

Software Development

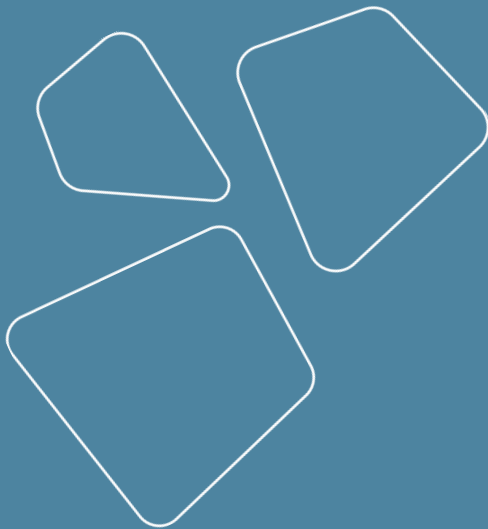
Vladislav Goncharenko

HSE, fall 2024



Timeline

- DRY, KISS, YAGNI
- SOLID
- Coupling/Cohesion
- Google Python Style Guide



YAGNI - You Aren't Gonna Need It

- Не пишите код, если не уверены, что он вам пригодится в будущем
- а.к.а “Бритва Оккама” -- не следует привлекать новые сущности без крайней на то необходимости
- (!) На этом курсе штрафуют за написанную, но неиспользованную функциональность. Кроме того штраф за:
 - Лишние файлы в репозитории (в тч лишние __init__.py)
 - Пустые файлы в репозитории (в тч пустой gitignore)
 - Закомментированный код (это красный флаг!!)

DRY - Don't Repeat Yourself

- Копипаста (дублирование) кода - это плохо
 - Сложно читать
 - Сложно изменять
- Если один и тот же код вызывается в двух и более местах - он должен быть обернут в функцию.
- (!) За дублирование кода штрафует на этом курсе.

KISS - Keep It Simple, Stupid

- Как правило, чем проще система, тем она:
 - надежнее
 - легче к прочтению и изменению
- То, что вы пишете сложный и перегруженный код, не делает вас хорошим разработчиком.
- Если вместо готовой реализации функции из какого-либо фреймворка вы пишете свою реализацию (без веских на то причин), то это плохо.
- (!) На этом курсе мы штрафует за “изобретение велосипеда”, и за сложный код (субъективное мнение проверяющего). В частности:
 - используйте pythonic style
 - используйте готовые реализации из фреймворков

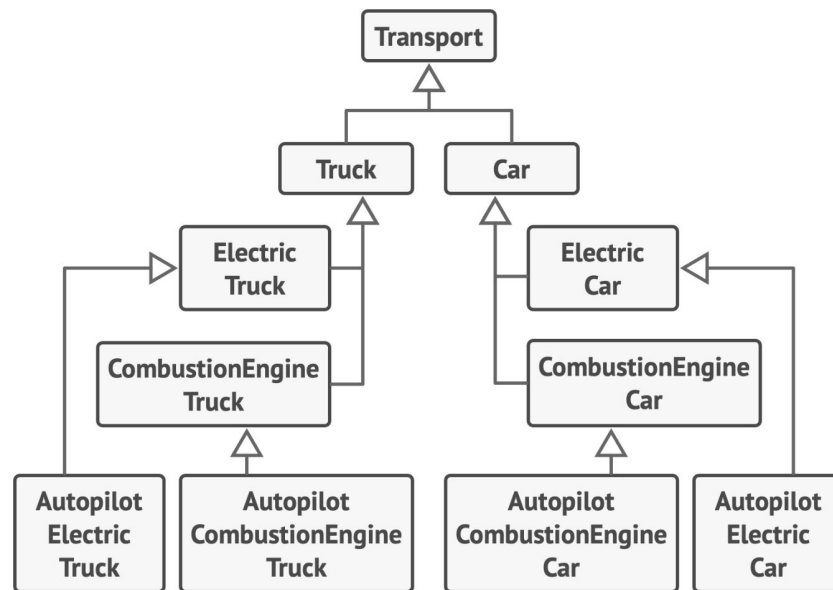
Forewords: Программируйте на уровне интерфейса

- Гибкость архитектуры определяется в том, чтобы ее можно было расширять, не ломая существующий код
- Это одно из самых базовых правил, позволяющих писать гибкий код

Forewords: Композиция как правило лучше наследования

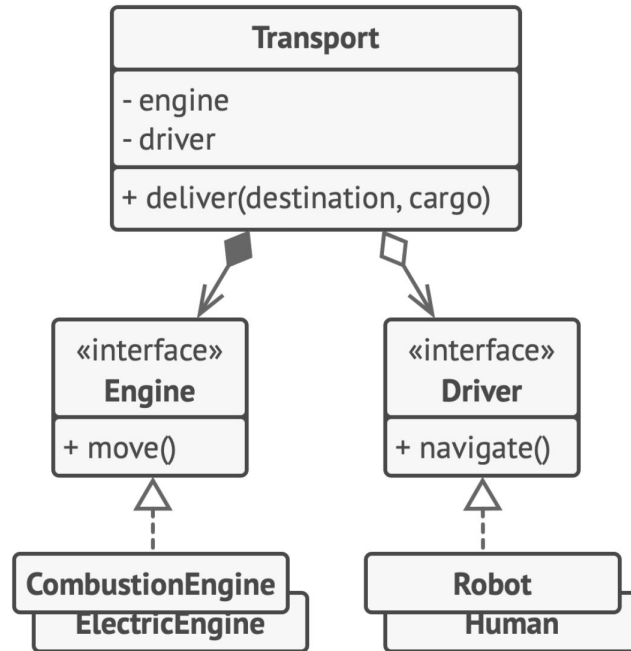
- При использовании наследования можете столкнуться с:
 - подкласс не может отказаться от интерфейса или реализации своего родителя
 - переопределяя методы родителя, вы должны быть предельно осторожны, чтобы не сломать базовое поведение, от которого зависят реализации подклассов
 - наследование нарушает инкапсуляцию суперкласса, так как подклассам доступны детали родителя

Forewords: Композиция как правило лучше наследования



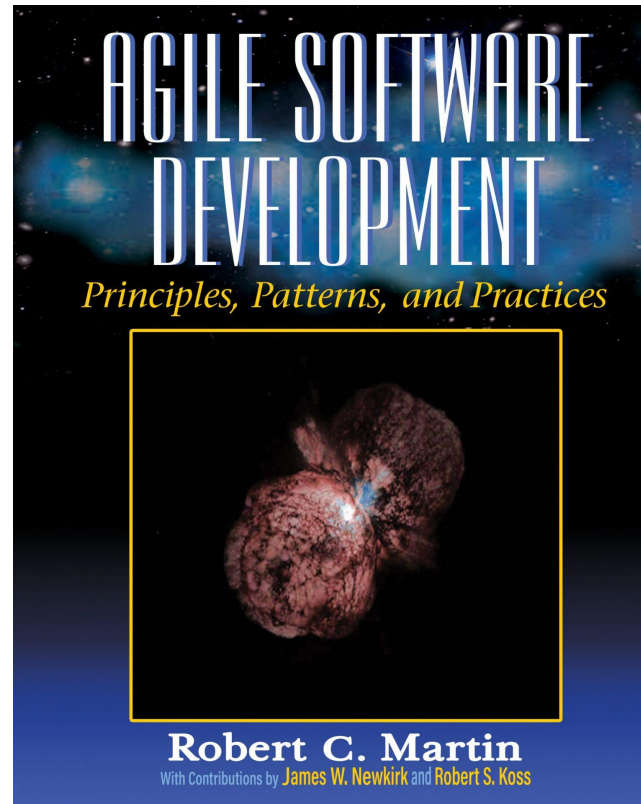
НАСЛЕДОВАНИЕ: развитие классов в нескольких плоскостях (тип груза × тип двигателя × тип навигации) приводит к комбинаторному взрыву.

Forewords: Композиция как правило лучше наследования



КОМПОЗИЦИЯ: различные виды функциональности выделены в собственные иерархии классов.

SOLID



(S) SINGLE RESPONSIBILITY PRINCIPLE (SPR)

Принцип единой ответственности

- У класса/функции должна быть только одна зона ответственности
- У класса/функции должен быть один и только один мотив для изменения
- Стремитесь к тому, чтобы каждый класс отвечал только за одну часть функциональности программы, причем желательно инкапсулировать эту функциональность только в этом классе
- Нужно стремиться к тому, чтобы иметь возможность сосредоточиться на сложных аспектах программы по отдельности, разделив их логически и физически

(O) OPEN/CLOSED PRINCIPLE

Принцип открытости/закрытости

- Расширяйте классы, но не изменяйте их первоначальный код

Типы классов:

- Класс можно назвать открытым, если он доступен для расширения
 - Например добавить новые операции / новые поля
- Класс можно назвать закрытым (законченным), если он готов для использования клиентским кодом

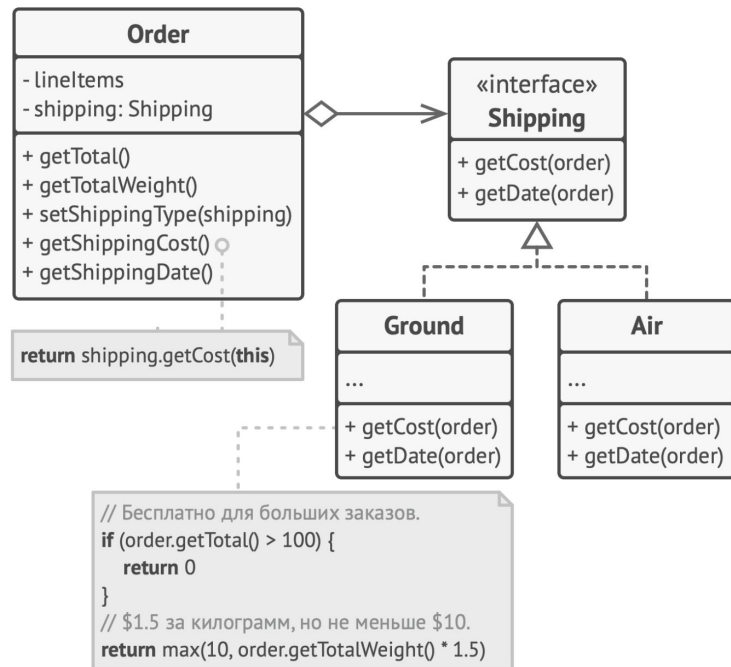
(O) OPEN/CLOSED PRINCIPLE



```
if (shipping == "ground") {  
    // Бесплатно для больших заказов.  
    if (getTotal() > 100) {  
        return 0  
    }  
    // $1.5 за килограмм, но не меньше $10.  
    return max(10, getTotalWeight() * 1.5)  
}  
  
if (shipping == "air") {  
    // $3 за килограмм, но не меньше $20.  
    return max(20, getTotalWeight() * 3)  
}
```

ДО: код класса заказа нужно будет изменять при добавлении нового способа доставки.

(O) OPEN/CLOSED PRINCIPLE



ПОСЛЕ: новые способы доставки можно добавить, не трогая класс заказов.

(L) LISKOV SUBSTITUTION PRINCIPLE

Принцип подстановки Лисков

- Подклассы должны дополнять, а не замещать поведение базового класса
- Чрезвычайно важно, если ваш код используется в качестве фреймворка
- Весь принцип - это ряд проверок, которые помогают ответить на вопрос: останется ли подкласс совместим с остальным кодом программы, который до этого корректно работал с объектами базового класса

(L) LISKOV SUBSTITUTION PRINCIPLE

Формализация требований:

- Типы параметров метода подкласса должны совпадать или быть более абстрактными, чем типы параметров базового метода
 - Базовый класс содержит метод **feed(Cat c)** , который умеет кормить домашних котов
 - **Хорошо**: Вы создали подкласс и переопределили метод кормёжки так, чтобы накормить любое животное: `feed(Animal c)`
 - **Плохо**: Вы создали другой подкласс, в котором метод умеет кормить только бенгальскую породу котов (под- класс котов): `feed(BengalCat c)`

(L) LISKOV SUBSTITUTION PRINCIPLE

Формализация требований:

- Тип возвращаемого значения метода подкласса должен **совпадать или быть подтипом** возвращаемого значения базового метода
 - Базовый метод: `buyCat(): Cat` . Клиентский код ожидает на выходе любого домашнего кота.
 - **Хорошо**: Метод подкласса: `buyCat(): BengalCat` . Клиентский код получит бенгальского кота, который является домашним котом, поэтому всё будет хорошо.
 - **Плохо**: Метод подкласса: `buyCat(): Animal` . Клиентский код сломается, так как это непонятное животное (возможно, крокодил) не поместится в ящике-переноске для кота.

(L) LISKOV SUBSTITUTION PRINCIPLE

Формализация требований:

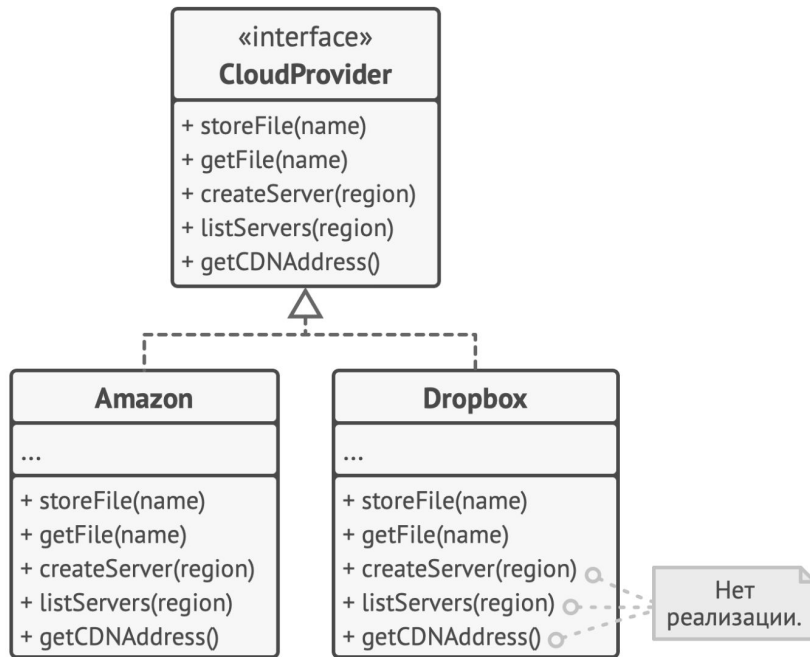
- Метод не должен выбрасывать исключения, которые не свойственны базовому методу.
- Метод не должен ужесточать условия на аргументы.

(I) INTERFACE SEGREGATION PRINCIPLE

Принцип разделения интерфейса

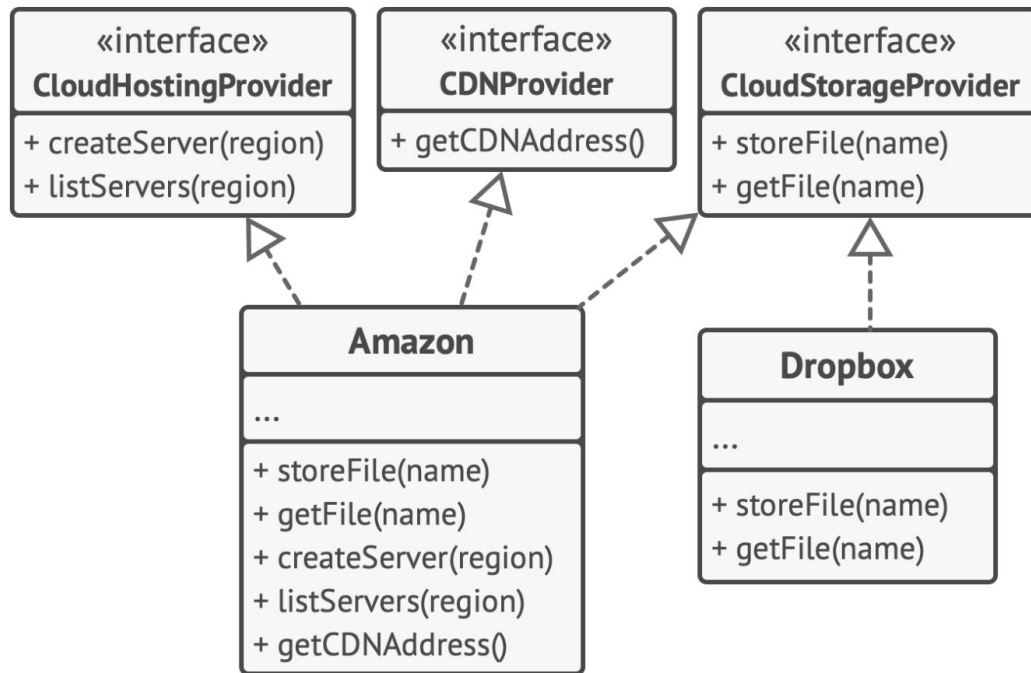
- Клиенты не должны зависеть от методов, которые они НЕ используют
- Нужно стремиться к тому, чтобы интерфейсы были достаточно узкими, чтобы класса не приходилось реализовывать избыточное поведение.

(I) INTERFACE SEGREGATION PRINCIPLE



ДО: не все клиенты могут реализовать операции интерфейса.

(I) INTERFACE SEGREGATION PRINCIPLE



ПОСЛЕ: раздутый интерфейс разбит на части.

(D) DEPENDENCY INVERSION PRINCIPLE

Принцип инверсии зависимостей

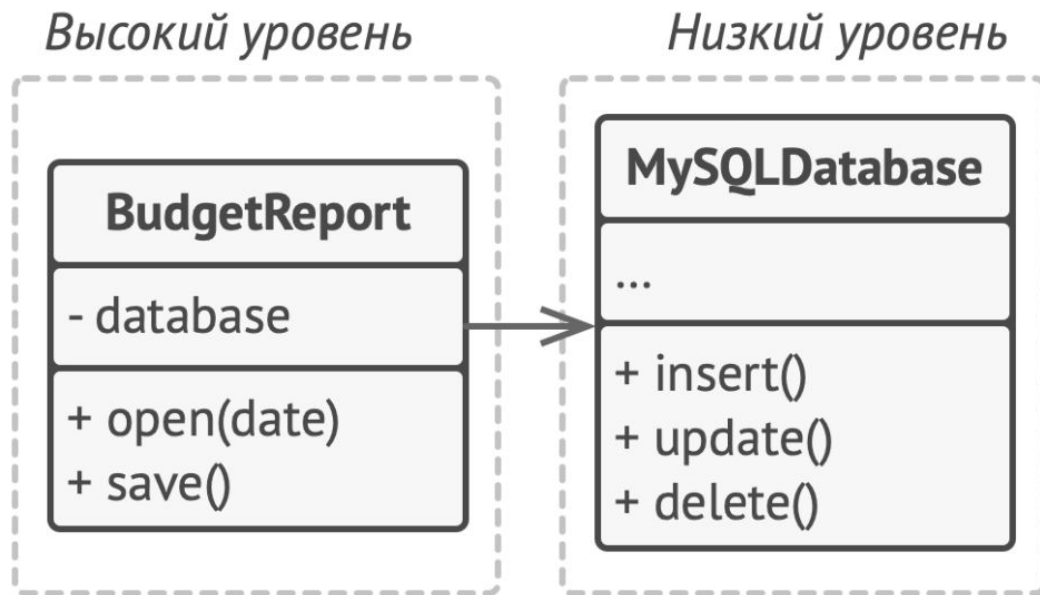
- Классы верхних уровней не должны зависеть от классов нижних уровней
- Классы верхних и нижних уровней должны зависеть от абстракций
- Абстракции не должны зависеть от деталей реализации
- Детали реализации должны зависеть от абстракций

(D) DEPENDENCY INVERSION PRINCIPLE

А как быть?

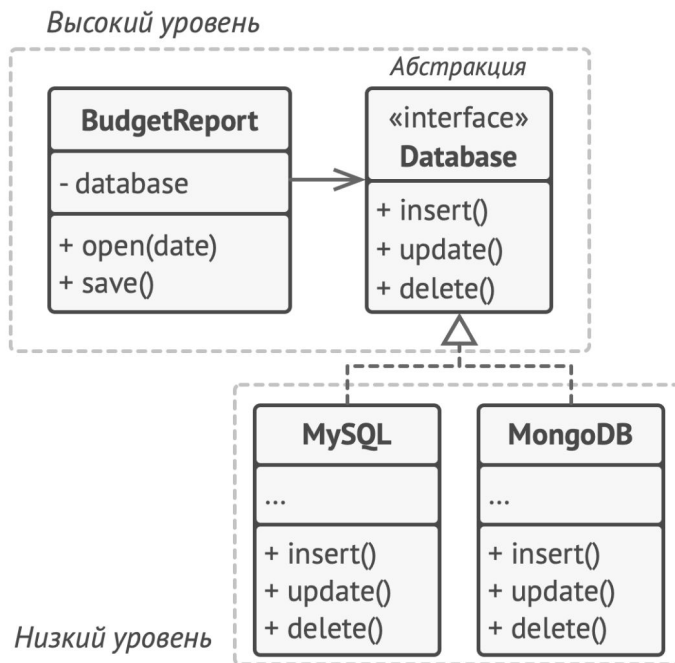
- Сперва нужно описать интерфейс низкоуровневых операций, которые нужны классам высокоуровневых операций
- Низкоуровневая логика в таком случае станет зависимой от интерфейса, который определяется высокоуровневой логикой

(D) DEPENDENCY INVERSION PRINCIPLE



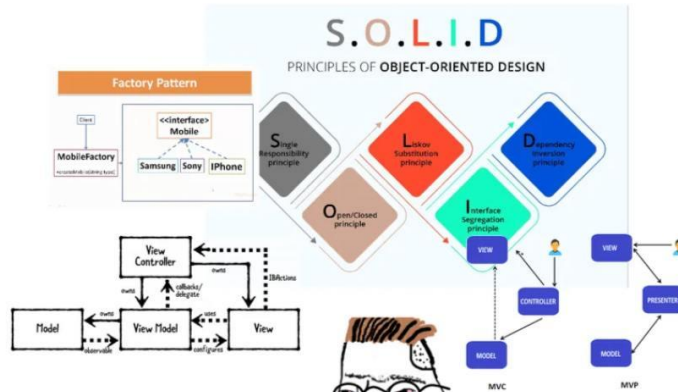
ДО: высокоуровневый класс зависит от низкоуровневого.

(D) DEPENDENCY INVERSION PRINCIPLE



ПОСЛЕ: низкоуровневые классы зависят от
высокоуровневой абстракции.

Real development



I will write only the code
needed to solve the problem

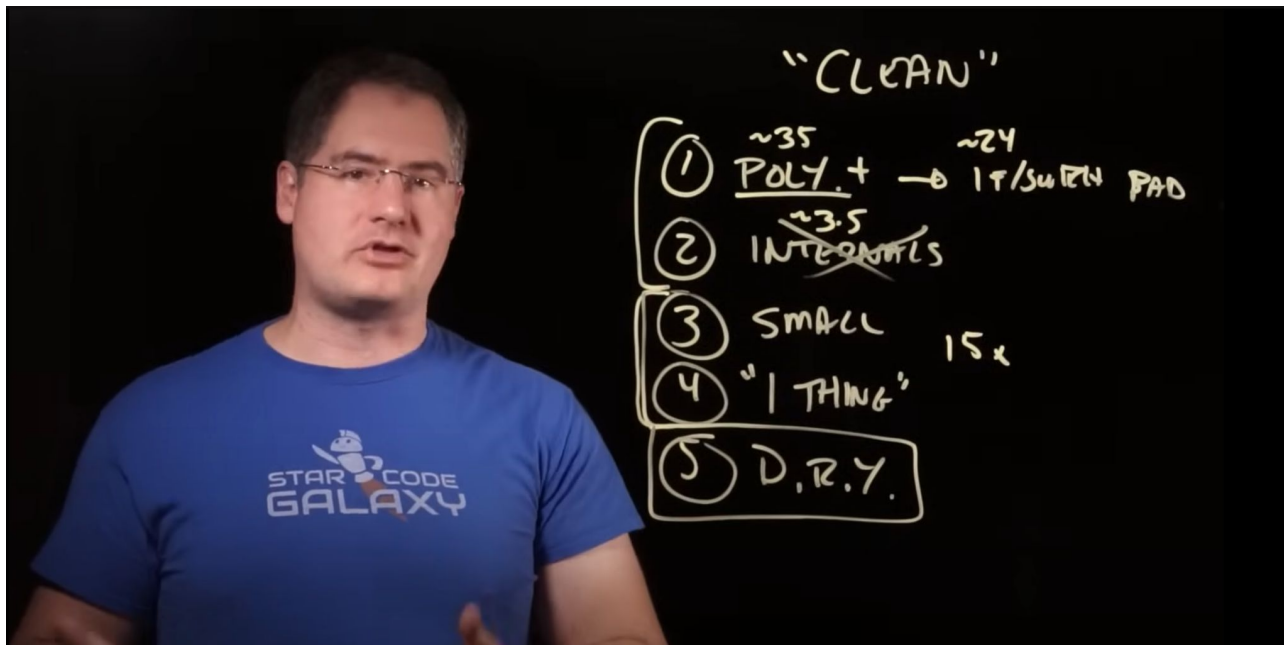


I will write only the code
needed to solve the problem



But still it's a long way...

Criticism



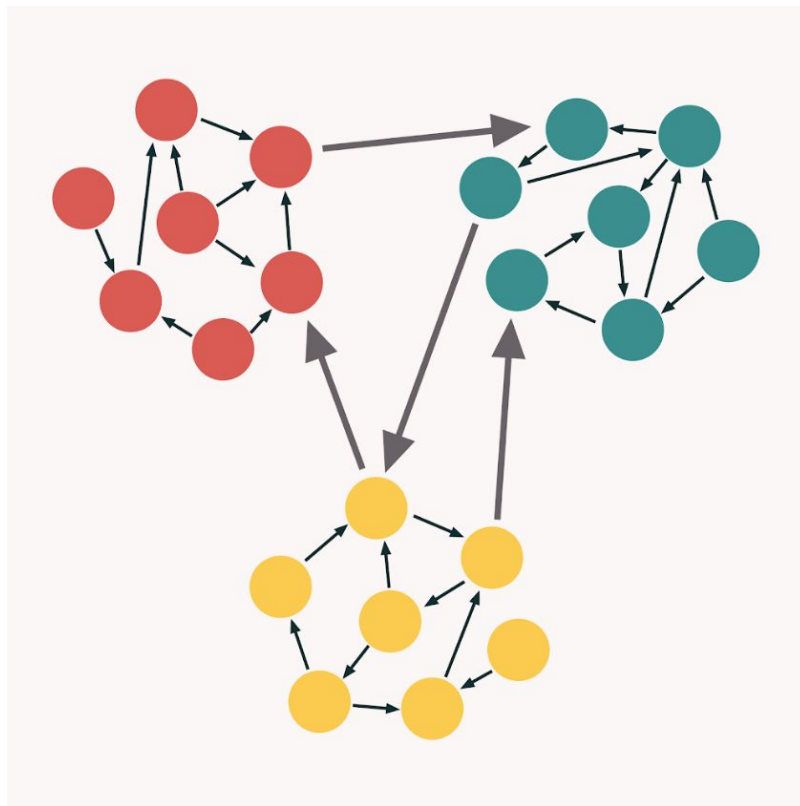
<https://www.youtube.com/watch?v=tD5NrevFtbU>

<https://www.youtube.com/watch?v=oFiY7EEtAW4> [ru]

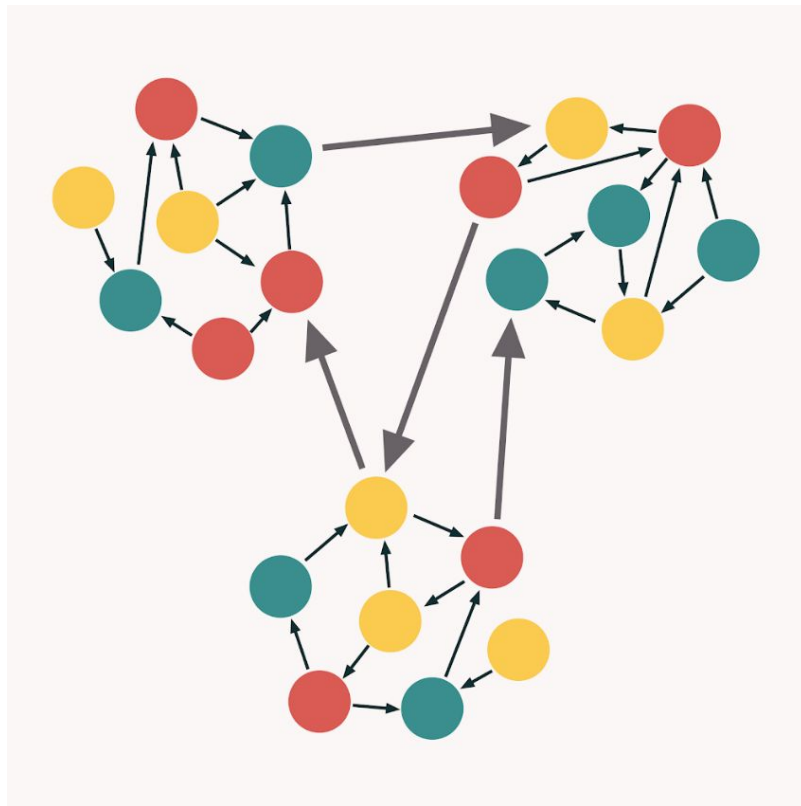
High Cohesion & Low Coupling

- **Cohesion** -- сцепление -- степень зависимости элементов внутри одного модуля. Модули с высоким сцеплением обычно:
 - Легче понять и поддерживать
 - Имеют меньше зависимостей от других модулей, что упрощает их изолирование
 - Способствуют лучшей организации кода
- **Coupling** -- связность -- Степень, в которой различные модули зависят друг от друга. Модули с низкой связностью:
 - Имеют меньше прямых зависимостей от других модулей, что упрощает изменения в одном модуле (нет необходимости менять другой модуль)
 - Улучшают повторное использование кода, так как каждый модуль может быть использован в разных контекстах без необходимости вносить изменения

Идеальный код: (High Cohesion & Low Coupling)



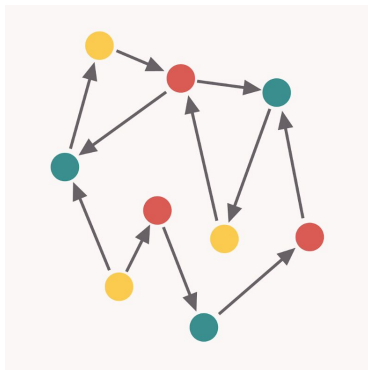
Плохо выбранные границы: (Low Cohesion & Low Coupling)



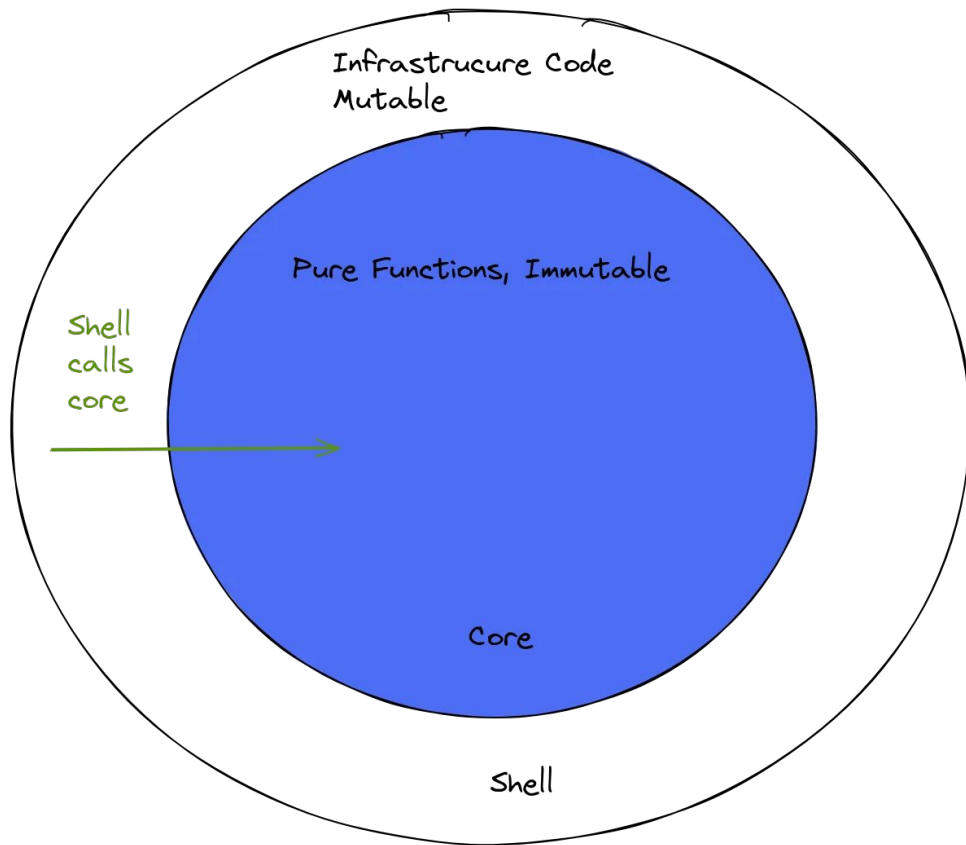
Cohesion & Coupling

Подведем итоги

- **Cohesion** - степень, в которой модуль представляет собой логически атомарную единицу
- **Coupling** - степень, в которой один модуль зависит от другого модуля
- Невозможно добиться минимального coupling (decoupling) без нарушения целостности (cohesion) и наоборот. Иначе получите деструктивный decoupling:



Functional Core and Imperative Shell Pattern



- <https://medium.com/ssense-tech/a-look-at-the-functional-core-and-imperative-shell-pattern-b0e2498da153a>
- <https://dev.to/cherryramatis/ending-the-war-or-continuing-it-lets-bring-functional-programming-to-oop-codebases-3mhd>

Google Pythonic Style

<https://google.github.io/styleguide/pyguide.html>

Спасибо за внимание!

Жду вопросов и обсуждений

